HOA SEN UNIVERSITY
RESPECT DIVERSITY

Con trỏ (Pointer)

Nhập môn lập trình

Vũ Đình Khôi

Nội dung

- Khái niệm
- Khai báo
- Vùng trỏ tới
- Cấp phát và giải phóng vùng nhớ
- Con trỏ và mảng 1 chiều
- Con trỏ và xâu ký tự

QUIZ lần 3

Khái niệm

- Biến tĩnh (static variable)
  - Biến có kích thước, kiểu của biến và địa chỉ của nó là không đổi.
  - Các biến này tồn tại trong suốt quá trình chạy chương trình.
- Biến động (dynamic variable)
  - Biến có thể thay đổi được kích thước, vùng nhớ và địa chỉ của vùng nhớ được cấp phát lúc chạy chương trình.
  - Có thể giải phóng biến động sau khi sử dụng để tiết kiệm bộ nhớ.

Khái niệm

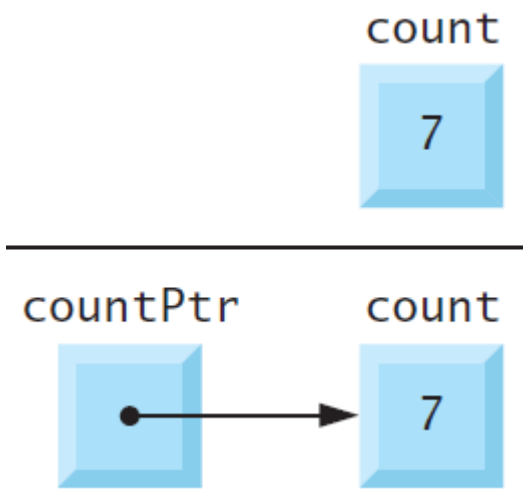
```c
/* Static variables & Dynamic variables */
#include <stdio.h>

void func() {
    static int x = 10; // static variable
    int y = 10; // dynamic variable
    printf("x = %d, y = %d\n", x++, y++);
}

void main() {
    int i;
    for (i = 0; i < 5; i++) {
        func();
    }
}
```

D:\Codes\C\static_variable.exe

```
x = 10, y = 10
x = 11, y = 10
x = 12, y = 10
x = 13, y = 10
x = 14, y = 10
```

Con trỏ hay biến con trỏ (pointer)

- Các biến bình thường
  - chứa trực tiếp một giá trị cụ thể nào đó (specific value). Ví dụ: int a= 5, b = 10;
- Con trỏ (hay biến con trỏ)
  - biến mà giá trị của nó là địa chỉ của một vùng nhớ (memory address). Vùng nhớ này có thể chứa giá trị cụ thể của một biến khác.



Biến con trỏ: ví dụ

Tên biến (Variable name)	num	ptr1	ptr2
Phép gán (Assignment)	num = 10	ptr1 = &num	ptr2 = &ptr1
Kiểu dữ liệu (Data type)	int	int*	int**
Giá trị (Value)	10	6487580	6487568
Địa chỉ vùng nhớ (Address)	6487580	6487568	6487560

D:\Codes\C\pointer_06.exe

```
ADDRESS                    VALUE
&num: 6487580              num: 10
&ptr1: 6487568             ptr1: 6487580
&ptr2: 6487560             ptr2: 6487568
```

Biến con trỏ: ví dụ

```c
/* Ban chat cua pointer, pointer-to-variable, pointer-to-pointer */
#include <stdio.h>

int main() {
    // Khai bao bien num kieu int
    int num = 10;
    printf("ADDRESS\t\t\t VALUE\n");
    printf("&num: %d\t\t num: %d\n", &num, num);

    // Khai bao con tro ptr1 kieu int tro den bien num
    int* ptr1;
    ptr1 = &num;
    printf("&ptr1: %d\t\t ptr1: %d\n", &ptr1, ptr1);

    // Khai bao con tro ptr2 kieu int tro den con tro ptr1
    int** ptr2;
    ptr2 = &ptr1;
    printf("&ptr2: %d\t\t ptr2: %d\n", &ptr2, ptr2);

    return 0;
}
```

Khai báo biến con trỏ

- Vì con trỏ cũng là một biến (mà giá trị của nó là địa chỉ của biến khác), nên trước khi sử dụng phải khai báo theo cú pháp sau:

data_type *pointer_name;

- Ví dụ

```
int *px;
float *ptr1, *ptr2;
char *s;
```

- Ý nghĩa
  - px là con trỏ tới kiểu int. Hay px là con trỏ tới đối tượng kiểu int
- Bản chất
  - px lưu giữ địa chỉ của một vùng nhớ có chiều dài 4 bytes (tương ứng với kiểu int). Ví dụ px = 100 → dữ liệu mà px trỏ đến sẽ nằm trong các ô nhớ liền kề có địa chỉ 100, 101, 102, 103

Gán giá trị cho biến con trỏ (vùng trỏ tới)

- Định vị con trỏ trỏ tới địa chỉ của một biến (*hay gán địa chỉ của một biến cho con trỏ*)

```
pointer_name = &variable_name;
```

- Ví dụ

```
int a = 10, b = 5;
int *px, *py;
float pi = 3.14;
float *m;
px = &a;
py = &b;
m = &pi;
```

- Lưu ý
  - Con trỏ và biến mà nó trỏ tới phải cùng kiểu dữ liệu.

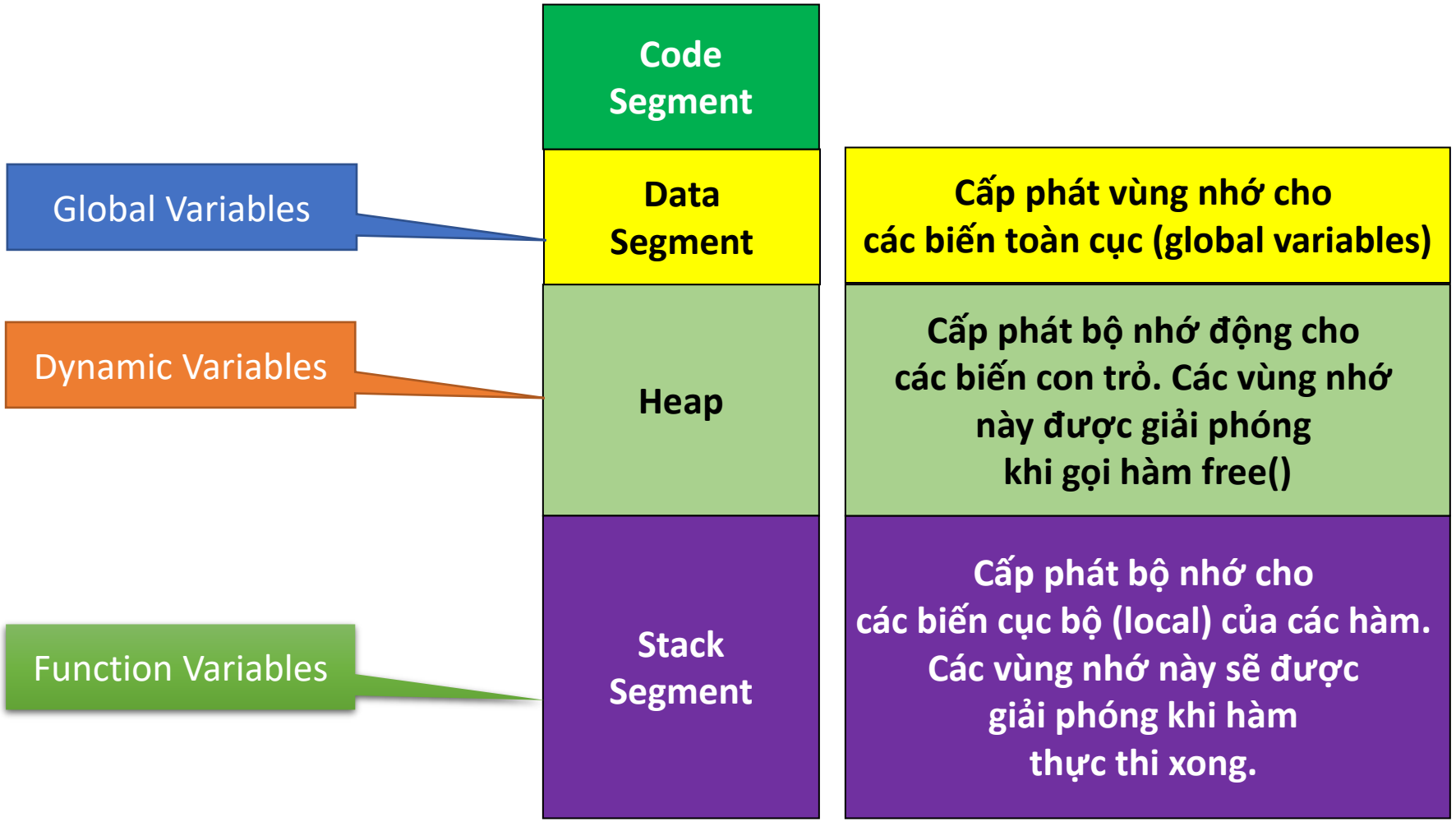
Ví dụ

```c
/* Ban chat cua pointer, pointer-to-variable
#include <stdio.h>

int main() {
    // Khai bao bien num kieu int
    int n = 10;
    int *p; // Khai bao con tro p kieu int
    p = &n; // Gan dia chi cua bien n cho con tro p

    printf("Gia tri cua bien n la: %d\n", n);
    printf("Dia chi cua bien n la: %d\n", &n);
    printf("Gia tri cua bien p la: %d\n", p);
    printf("Gia tri cua bien p tro toi: %d\n", *p);
    printf("Dia chi cua bien p la: %d\n", &p);

    *p = 20; // Gan gia tri 20 cho bien n
    printf("Gia tri cua bien n la: %d\n", n);

    return 0;
}
```

D:\Codes\C\pointer_07.exe

```
Gia tri cua bien n la: 10
Dia chi cua bien n la: 6487580
Gia tri cua bien p la: 6487580
Gia tri cua bien p tro toi: 10
Dia chi cua bien p la: 6487568
Gia tri cua bien n la: 20
```

Bộ nhớ được tổ chức như thế nào?



```c
/* Memory Allocation */
#include <stdio.h>
#include <stdlib.h>
// Global variables
int MAX = 100;
// Function definition
int sum(int a, int b) {
    int s;
    printf("Address of a in sum: %u\n", &a);
    printf("Address of b in sum: %u\n", &b);
    printf("Address of s in sum: %u\n", &s);
    s = a + b;
    return s;
}
int main() {
    int x = 5, y = 10;
    int *p = (int*) malloc(sizeof(int));
    printf("Address of x in main: %u\n", &x);
    printf("Address of y in main: %u\n", &y);
    printf("Address of p in main: %u\n", &p);
    printf("Address of main(): %u\n", &main);
    printf("Address of sum(): %u\n", &sum);
    printf("Address of MAX: %u\n", &MAX);
    printf("Sum = %d\n", sum(x, y));
    free(p);
    return 0;
}
```

DATA SEGMENT

MAX = 100

CODE SEGMENT

Code of sum()
Code of main()

STACK SEGMENT

a, b, s
x, y

HEAP

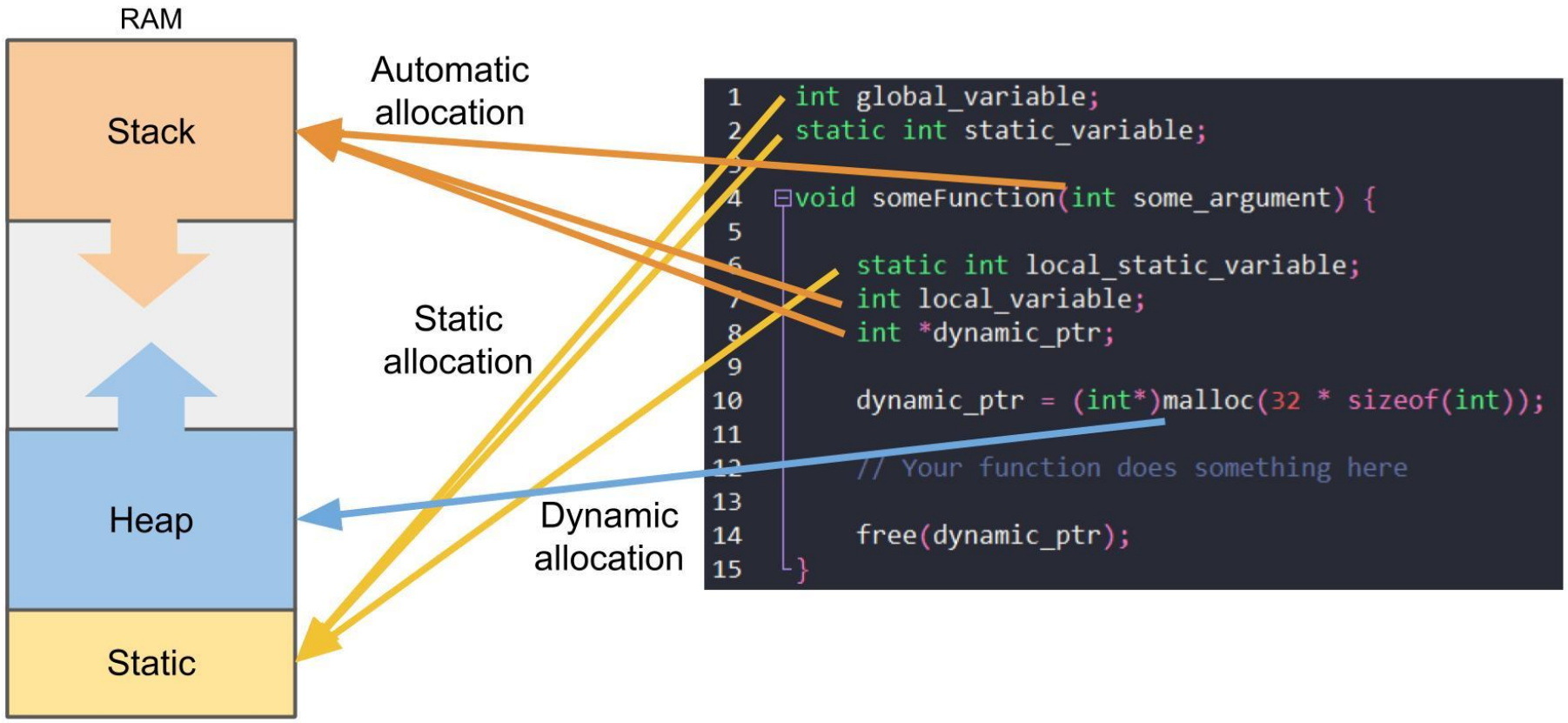
p

D:\Codes\C\memory_allocation.exe

```
Address of x in main: 6487580
Address of y in main: 6487576
Address of p in main: 6487568
Address of main(): 4199816
Address of sum(): 4199728
Address of MAX: 4206608
Address of a in sum: 6487536
Address of b in sum: 6487544
Address of s in sum: 6487516
Sum = 15
```

Bộ nhớ được tổ chức như thế nào?

Memory Allocation




```c
int global_variable;
static int static_variable;

void someFunction(int some_argument) {

    static int local_static_variable;
    int local_variable;
    int *dynamic_ptr;

    dynamic_ptr = (int*)malloc(32 * sizeof(int));

    // Your function does something here

    free(dynamic_ptr);
}
```

Stack và Heap

Stack	Heap
• Vùng nhớ được cấp phát khi chương trình được biên dịch.	• Vùng nhớ được cấp phát khi chạy chương trình (run-time).
• Vùng nhớ stack được sử dụng cho việc thực thi thread. Khi gọi hàm, các biến cục bộ của hàm được lưu trữ vào block của stack (theo kiểu LIFO). Cho đến khi hàm trả về giá trị, block này sẽ được xóa tự động.	• Vùng nhớ heap được dùng cho cấp phát bộ nhớ động (malloc(), từ khóa new). Vùng nhớ được cấp phát tồn tại đến khi giải phóng vùng nhớ bằng lệnh free() hoặc delete().
• Kích thước vùng nhớ stack được cố định. Chúng ta không thể tăng hoặc giảm kích thước vùng nhớ stack. Nếu không đủ vùng nhớ stack, gây ra stack overflow. Hiện tượng này xảy ra khi nhiều hàm lồng nhau hoặc đệ quy nhiều lần dẫn đến không đủ vùng nhớ.	• Khi kích thước vùng nhớ heap không đủ cho yêu cầu malloc(), new. Hệ điều hành sẽ có cơ chế tăng kích thước vùng nhớ heap.

Nguồn: http://vncoding.net

Cấp phát (allocate) & giải phóng (free) vùng nhớ

- Thông thường, chúng ta chỉ quan tâm đến khai báo và cung cấp vùng nhớ để lưu địa chỉ của biến mà không quan tâm đến việc cung cấp vùng nhớ để lưu dữ liệu.
- Cấp phát vùng nhớ
  - `pointer = malloc(sizeof(data_type));`
  - `pointer = malloc(n * sizeof(data_type));`
  - `pointer = calloc(n, sizeof(data_type));`
- Giải phóng vùng nhớ
  - `free(pointer);`

Cấp phát (allocate) & giải phóng (free) vùng nhớ

- Khai báo và cấp phát vùng nhớ cho biến con trỏ n

```
int *n = (int*)malloc(sizeof(int));
```

- Khai báo và cấp phát vùng nhớ cho biến trỏ a là một mảng có 20 phần tử

```
int *a = (int*)malloc(20 * sizeof(int));
```

- Hoặc

```
int *a = (int*)calloc(20, sizeof(int));
```

- Giải phóng vùng nhớ

```
free(n);
free(a);
```

Cấp phát (allocate) & giải phóng (free) vùng nhớ

```c
/* Memory Allocation */
#include <stdio.h>
#include <malloc.h>  // hoac <stdlib.h>

int main() {
    char *p;
    p = (char *)malloc(sizeof(char));
    p = "Hello students!";
    printf("%s\n", p);
    free(p);

    return 0;
}
```

Cấp phát vùng nhớ

Giải phóng vùng nhớ

Cấp phát (new) & giải phóng (delete) vùng nhớ

- Do lập trình trên môi trường C++ nên có thể dùng từ khoá new và hàm delete() để cấp phát và giải phóng vùng nhớ.
- Khai báo:

```
int *n = new int;
int *a = new int[20];
```

- Giải phóng vùng nhớ:

```
delete(n);
delete(a);
```

Cấp phát (new) & giải phóng (delete) vùng nhớ

```
1   /* Memory Allocation: memory_allocation.cpp */
2   #include <stdio.h>
3   #include <malloc.h>  // hoac <stdlib.h>
4
5   int main() {
6       int x = 5;
7       // Cap phat vung nho cho bien con tro
8       int *n = (int*)malloc(sizeof(int));  // theo C, dung malloc va free
9       int *m = new int;  // theo C++, dung new va delete
10      n = &x;
11      m = &x;
12      printf("Address of x: %u\n", &x);
13      printf("Value of n: %u\n", n);
14      printf("Value of m: %u\n", m);
15      printf("Value of x: %u\n", *n);
16      // Giai phong vung nho
17      free(n);
18      delete(m);
19      return 0;
20  }
```

D:\Codes\C\memory_allocation_2.exe

```
Address of x: 6684172
Value of n: 6684172
Value of m: 6684172
Value of x: 5
```

Cấp phát (new) & giải phóng (delete) vùng nhớ

```
1   /* Memory Allocation: memory_allocation.cpp */
2   #include <stdio.h>
3   #include <malloc.h>  // hoac <stdlib.h>
4
5   int main() {
6       int n = 5;
7       // Cap phat vung nho cho bien con tro
8       int *arr1 = (int*)malloc(n * sizeof(int));  // theo C, dung malloc va free
9       int *arr2 = new int[n];  // theo C++, dung new va delete
10      // Khoi tao gia tri
11      for (int i = 0; i < n; i++) {
12          arr1[i] = i;
13          arr2[i] = i * i;
14      }
15
16      for (int i = 0; i < n; i++) {
17          printf("arr1[%d] = %d\t", i, arr1[i]);
18          printf("arr2[%d] = %d\n", i, arr2[i]);
19      }
20      // Giai phong vung nho
21      free(arr1);
22      delete(arr2);
23
24      return 0;
25  }
```

```
D:\Codes\C\memory_allocation_3.exe
arr1[0] = 0     arr2[0] = 0
arr1[1] = 1     arr2[1] = 1
arr1[2] = 2     arr2[2] = 4
arr1[3] = 3     arr2[3] = 9
arr1[4] = 4     arr2[4] = 16
```

Con trỏ và mảng 1 chiều

- Phép toán lấy địa chỉ của các phần tử trong mảng

```
float a[10];
&a[i] sẽ là địa chỉ của phần tử a[i], với i trong khoảng [0, 9]
```

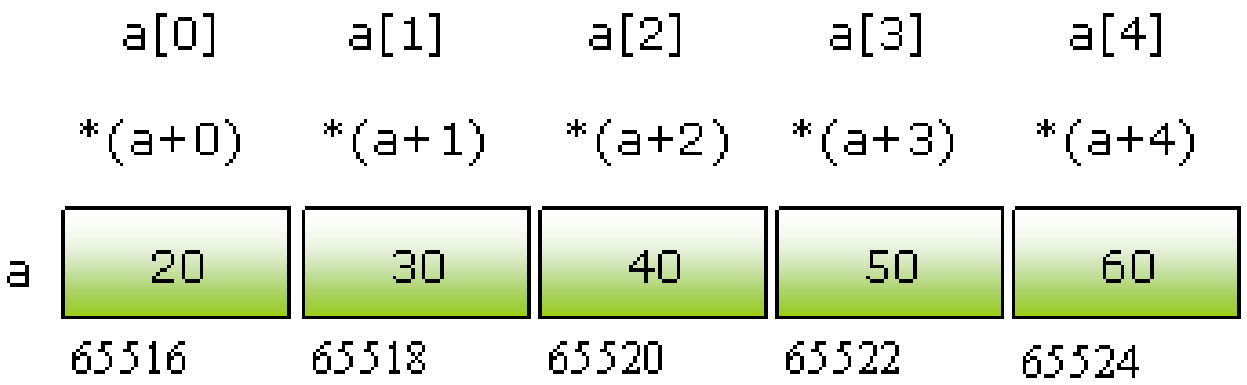
- Cách lấy giá trị của phần tử trong mảng

```
float a[10], *p;
p = a;
```

- Các cách viết sau đây là tương đương nhau

```
a[i]      *(a + i)  *(p + i)          p[i]
```

Con trỏ và mảng 1 chiều



*(a+0)	value at 65516	20	a[0]
*(a+1)	value at 65518	30	a[1]
*(a+2)	value at 65520	40	a[2]
*(a+3)	value at 65522	50	a[3]

-

-

*(a+i) (Or) a[i]

Khai báo mảng 1 chiều và con trỏ

- Thông thường ta khai báo

```
int a[20];
float b[20];
double c[20];
```

- Ta khai báo kiểu con trỏ như sau

```
int *a;
float *b;
double *c;
```

Truy xuất các phần tử trong mảng

```c
int main() {
    int n = 5, a[n], *p;
    p = a;
    int i;
    for (i = 0; i < n; i++)
        a[i] = i * i;

    for (i = 0; i < n; i++)
        printf("%3d,", a[i]);
    printf("\n");

    for (i = 0; i < n; i++)
        printf("%3d,", *(a + i));  // vi a la bien con tro
    printf("\n");

    for (i = 0; i < n; i++)
        printf("%3d,", p[i]);
    printf("\n");

    for (i = 0; i < n; i++)
        printf("%3d,", *(p + i));
    printf("\n");

    return 0;
}
```

Con trỏ và xâu/chuỗi (string) ký tự

- Xem 2 cách khai báo sau

```
char name[30]; //Khai báo biến name là 1 chuỗi tối đa 30 ký tự
char *p_name; //Khai báo biến con trỏ p_name kiểu char
```

- Nếu ta sử dụng các phép gán sau

```
name = “Nguyen Van A”;  // Không chấp nhận
p_name = “Nguyen Van B”;  // Chấp nhận
```

Bài tập

- 1. Viết chương trình nhập tên của 1 sinh viên, in ra tên là chữ viết hoa và cắt bỏ tất cả khoảng trắng (sử dụng con trỏ).
- 2. Sử dụng con trỏ để viết chương trình thực hiện các công việc sau trên một mảng số nguyên: (mỗi công việc là 1 hàm)
  - Nhập mảng
  - Xuất mảng
  - Sắp xếp mảng
  - Tìm phần tử âm lớn nhất của mảng
  - Tìm phần tử dương bé nhất của mảng
  - Tìm những phần tử là số nguyên tố